ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN



NHẬN DẠNG MẪU

ĐỒ ÁN GIỮA KÌ

NHẬN DẠNG CHỮ SỐ VIẾT TAY BẰNG CÁC PHƯƠNG PHÁP PHÂN LỚP

Sinh viên thực hiện:

1. TRẦN HÙNG LUYỆN MSSV: 24110201
2. LÊ MỸ TÀI MSSV: 24110235

TP. HỒ CHÍ MINH, THÁNG 11/2026

Mục lục

Mã nguồn

1 Giới thiệu **3**
- 1.1 Bài toán và động cơ thực hiện . . . 3
- 1.2 Mục tiêu và phạm vi . . . 3

2 Dữ liệu và tiền xử lý **3**
- 2.1 Nạp dữ liệu và phân chia thực nghiệm . . . 3
- 2.2 Biểu diễn ảnh và tiền xử lý . . . 4
- 2.3 Phân bố lớp và biến thiên hình dạng . . . 4

3 Phương pháp **5**
- 3.1 Gaussian Naive Bayes . . . 5
- 3.2 Softmax Regression . . . 6
- 3.3 Mạng nhiều lớp MLP bằng NumPy . . . 7
- 3.4 Stacking với AdaBoost làm meta-learner . . . 8

4 Thiết kế thực nghiệm **9**
- 4.1 Giao thức đánh giá . . . 10
- 4.2 Độ đo phân lớp . . . 10
- 4.3 Đo thời gian và đối chiếu thư viện . . . 11

5 Kết quả thực nghiệm **11**
- 5.1 Ảnh hưởng của siêu tham số trên validation . . . 11
- 5.2 Kết quả cuối trên train, validation và test . . . 14
- 5.3 Lỗi theo lớp và đối chiếu dự đoán từng ảnh . . . 14
- 5.4 Đóng góp của voting và tầng AdaBoost . . . 16
- 5.5 Đối chiếu scikit-learn . . . 17
- 5.6 So sánh CNN và MLP có số tham số gần nhau . . . 17
- 5.7 Augmentation và độ bền khi ảnh bị biến đổi . . . 18

6 Thảo luận **19**
- 6.1 Đánh đổi độ chính xác, thời gian và khả năng giải thích . . . 19

7 Kết luận **20**

Phân công công việc **21**

Tài liệu tham khảo **21**

Quy ước. Train là tập huấn luyện, validation là tập chọn cấu hình và test là tập kiểm thử. P, R và F1 lần lượt là precision, recall và F1-score. Giá trị macro là trung bình của độ đo trên 10 lớp, với trọng số bằng nhau. Các độ đo được ghi theo thang $[0, 1]$ hoặc theo phần trăm, tùy ký hiệu trong từng bảng. Thời gian tính bằng giây (s). Chênh lệch giữa hai tỷ lệ phần trăm được ghi bằng điểm phần trăm (đpt).

TÓM TẮT

Đồ án phân loại chữ số viết tay trên MNIST bằng bốn phương pháp. Nhóm tự cài đặt Gaussian Naive Bayes, Softmax Regression và MLP bằng NumPy. Với phương pháp stacking, nhóm dùng dự đoán ngoài fold của các mô hình cơ sở để huấn luyện AdaBoost ở tầng thứ hai. Dữ liệu gồm 50.000 ảnh huấn luyện, 10.000 ảnh validation và 10.000 ảnh kiểm thử.

Trên tập kiểm thử, độ chính xác của Gaussian Naive Bayes, Softmax Regression, MLP và stacking lần lượt là 77,53%, 92,53%, 98,19% và 98,54%. Stacking dự đoán đúng hơn soft voting 2 ảnh trên tổng số 10.000 ảnh. Trong phần mở rộng, nhóm so sánh CNN với MLP có số tham số gần nhau và thử tăng cường dữ liệu. Với cấu hình đã thử, tăng cường dữ liệu giúp mô hình nhận dạng tốt hơn khi ảnh bị dịch chuyển hoặc xoay nhiều, nhưng làm giảm độ chính xác trên ảnh kiểm thử gốc.

1 Giới thiệu

1.1 Bài toán và động cơ thực hiện

Bài toán của đồ án là dự đoán chữ số từ một ảnh xám kích thước 28×28 pixel. Mỗi ảnh có một nhãn thuộc tập $\mathcal{C} = \{0, 1, \dots, 9\}$. Cùng một chữ số có thể được viết nghiêng, đậm hoặc lệch vị trí. Một số cách viết của hai chữ số khác nhau cũng khá giống nhau, chẳng hạn số 4 và số 9. Nhóm dùng MNIST để xem các phương pháp phân lớp xử lý những trường hợp này như thế nào.

1.2 Mục tiêu và phạm vi

Với Gaussian Naive Bayes, Softmax Regression và MLP, nhóm tự cài đặt phần huấn luyện và dự đoán bằng NumPy, sau đó đối chiếu kết quả với scikit-learn. Phần khảo sát dùng tập validation để chọn cấu hình. Nhóm đánh giá các cấu hình đã chọn trên tập kiểm thử.

Ở phần ensemble, nhóm khảo sát các mô hình cơ sở trên tập validation rồi chọn một bộ ba để thực hiện soft voting và stacking. Các mô hình cơ sở và AdaBoost sử dụng scikit-learn. Nhóm viết quy trình tạo dự đoán ngoài fold, ghép xác suất dự đoán và huấn luyện mô hình tầng thứ hai. Kết quả stacking được so sánh với soft voting và mô hình đơn được chọn trên validation.

Phần mở rộng dùng PyTorch để so sánh CNN và MLP260 với số tham số gần nhau. Nhóm thử tăng cường dữ liệu và đánh giá trên ảnh gốc, ảnh dịch chuyển và ảnh xoay. Ứng dụng Streamlit minh họa cách các mô hình dự đoán chữ số do người dùng vẽ hoặc tải lên.

2 Dữ liệu và tiền xử lý

2.1 Nạp dữ liệu và phân chia thực nghiệm

MNIST gồm 70.000 ảnh xám kích thước 28×28: 60.000 ảnh train gốc và 10.000 ảnh test gốc. Module `src/data.py` tải bốn file IDX nén gzip khi chưa có dữ liệu, đọc header và khôi phục mảng ảnh, nhãn. Hàm nạp trả về tập train và test gốc. Các notebook chia tập train gốc thành train và validation.

Phép chia dùng `train_test_split(test_size=10000, stratify=y, random_state=42)`. Chia phân tầng giúp duy trì tỷ lệ lớp gần dữ liệu gốc. Trong notebook 05, 50.000 ảnh train được chia thành 40.000 ảnh để huấn luyện và 10.000 ảnh để khảo sát siêu tham số của các mô hình nền.

Bảng 1
Phân chia dữ liệu và vai trò trong thực nghiệm.

Tập	Số ảnh	Vai trò
Train	50.000	Học tham số; tạo đặc trưng OOF cho stacking.
Validation	10.000	Khảo sát cấu hình, chọn checkpoint, bộ ba và trọng số voting.
Test	10.000	Đánh giá cấu hình cuối trên tập test gốc.

Nhóm dùng tập validation ngoài để so sánh mô hình và chọn bộ ba.

Các mô hình dùng cùng dữ liệu MNIST và cùng thiết lập chia phân tầng với seed 42. Ba tập train, validation và test lần lượt có 50.000, 10.000 và 10.000 ảnh.

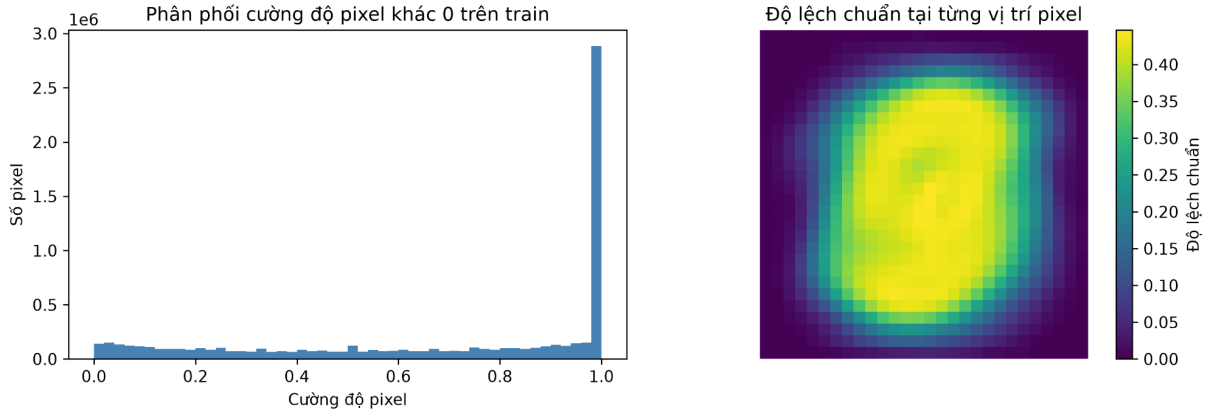
2.2 Biểu diễn ảnh và tiền xử lý

Ảnh được chuyển sang `float32` và chia cường độ pixel cho 255 để đưa về khoảng $[0, 1]$. Mỗi ảnh được trải phẳng thành vector 784 phần tử cho các mô hình dùng đầu vào dạng vector:

$$X_{ij} = I_{ij}/255, \qquad X \in \mathbb{R}^{N\times 784}, \quad y_i \in \mathcal{C}. \tag{1}$$

Nhãn được giữ dưới dạng số nguyên từ 0 đến 9. Với CNN, đầu vào có kích thước $(N, 1, 28, 28)$. Trong thí nghiệm augmentation, nhóm chỉ biến đổi ảnh train. Tập validation giữ ảnh gốc để chọn checkpoint, còn tập clean test dùng ảnh gốc để đánh giá.

Hình 1 mô tả cường độ pixel sau khi chuẩn hóa trên tập train. Biểu đồ bên trái chỉ đếm các pixel khác 0 và cho thấy cường độ tập trung mạnh ở gần 1. Bản đồ bên phải cho thấy cường độ pixel biến thiên nhiều hơn ở vùng giữa ảnh, nơi các nét chữ thường xuất hiện, và ít biến thiên ở vùng rìa.

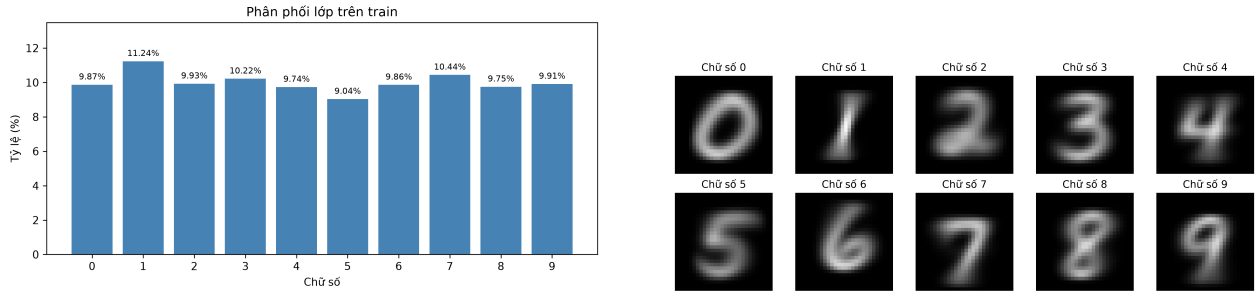


Hình 1
Phân bố cường độ pixel khác 0 (trái) và độ lệch chuẩn tại từng vị trí pixel (phải) trên tập train.

2.3 Phân bố lớp và biến thiên hình dạng

Trong 50.000 ảnh train, lớp 1 có 5.618 ảnh (11,236%), nhiều nhất; lớp 5 có 4.518 ảnh (9,036%), ít nhất. Chênh lệch này chưa quá lớn nhưng các lớp không hoàn toàn cân bằng. Vì vậy, accuracy được báo cáo cùng precision, recall và F1 macro, thay vì chỉ dùng một tỷ lệ đúng chung.

Ảnh trung bình cho thấy những vùng thường có nét trong từng lớp, nhưng làm mờ đặc điểm



Hình 2
Phân bố lớp trên train (trái) và ảnh trung bình theo lớp (phải).

riêng của từng cách viết. Khoảng cách Euclid giữa hai ảnh trung bình phản ánh sự khác biệt về hình dạng chung của hai lớp. Trong ma trận khoảng cách, cặp 4–9 gần nhau nhất (2,483), tiếp theo là 7–9 (2,988) và 3–5 (3,108). Mức độ nhầm lẫn giữa các lớp được đánh giá qua kết quả dự đoán trên tập test.

Các pixel lân cận thường cùng tạo thành nét chữ nên có quan hệ với nhau. Giả định độc lập có điều kiện của Gaussian NB vì vậy có thể hạn chế khả năng mô tả ảnh. CNN dùng bộ lọc tích chập để khai thác mối liên hệ giữa các pixel lân cận và nhận biết các đặc trưng như nét thẳng và nét cong.

3 Phương pháp

3.1 Gaussian Naive Bayes

Nhóm dùng Gaussian Naive Bayes để phân loại ảnh MNIST dựa trên 784 giá trị pixel. Với mỗi lớp, mô hình giả định giá trị của từng pixel tuân theo phân phối Gaussian và các pixel độc lập với nhau khi đã biết lớp [1, 2]. Gọi n_c là số ảnh thuộc lớp c trong N ảnh train và x_{ij} là giá trị pixel thứ j của ảnh thứ i. Các tham số được tính như sau:

$$\pi_c = n_c/N, \qquad \mu_{cj} = \frac{1}{n_c} \sum_{i:y_i=c} x_{ij}, \tag{2}$$

$$\tilde{\sigma}^2_{cj} = \frac{1}{n_c} \sum_{i:y_i=c} (x_{ij} - \mu_{cj})^2 + \varepsilon. \tag{3}$$

Trong bản tự cài đặt, nhóm tính phương sai theo từng pixel với mẫu số n_c và cộng trực tiếp ε để tránh chia cho 0 ở những pixel có phương sai bằng 0. Tham số `var_smoothing` trong `src/models/gaussian.py` chính là ε. Với scikit-learn, lượng cộng bằng `var_smoothing` nhân với phương sai lớn nhất trong 784 đặc trưng, tính trên toàn bộ tập train.

Khi dự đoán, nhóm cộng các log mật độ Gaussian thay cho phép nhân các mật độ để hạn chế underflow. Điểm $g_c(\mathbf{x})$ là log của tích giữa xác suất tiên nghiệm của lớp c và mật độ của ảnh $\mathbf{x}$ trong lớp đó. Nhãn dự đoán là lớp có điểm cao nhất.

$$g_c(\mathbf{x}) = \log \pi_c - \frac{1}{2} \sum_{j=1}^{784} \left[\log(2\pi\tilde{\sigma}^2_{cj}) + \frac{(x_j - \mu_{cj})^2}{\tilde{\sigma}^2_{cj}} \right], \qquad \hat{y} = \arg\max_c g_c(\mathbf{x}). \tag{4}$$

Mẫu số trong công thức Bayes giống nhau cho mọi lớp của cùng một ảnh, nên có thể chọn nhãn trực tiếp từ $g_c(\mathbf{x})$.

Mã giả 1. Gaussian NB tự cài đặt

1. Đầu vào: X, y, ε. Xác định tập lớp $\mathcal{C} \leftarrow \{y_i\}_{i=1}^{N}$.
2. Với mỗi $c \in \mathcal{C}$: $X_c \leftarrow X[y = c]$, $n_c \leftarrow |X_c|$, $\pi_c \leftarrow n_c/N$.
3. Theo từng pixel: $\mu_c \leftarrow \text{mean}(X_c)$, $\widetilde{\sigma}_c^2 \leftarrow \text{var}(X_c) + \varepsilon$.
4. Với ảnh $\mathbf{x}$, tính $g_c(\mathbf{x})$ theo (4) cho từng $c \in \mathcal{C}$. Trả về $\widehat{y} \leftarrow \arg\max_{c \in \mathcal{C}} g_c(\mathbf{x})$.

Mô hình lưu 10 xác suất tiên nghiệm, 10×784 giá trị trung bình và 10×784 phương sai, tổng cộng 15.690 giá trị. Khi đánh giá, notebook chia ảnh thành các lô tối đa 2.048 mẫu để giảm bộ nhớ tạm lúc tính điểm cho 10 lớp.

3.2 Softmax Regression

Với một mini-batch gồm B ảnh, $X \in \mathbb{R}^{B \times 784}$ chứa một ảnh trên mỗi hàng. Mô hình dùng ma trận trọng số $W \in \mathbb{R}^{784 \times 10}$ và vector bias cột $\mathbf{b} \in \mathbb{R}^{10}$. Trong NumPy, phép tính `X @ W + b` cộng cùng một bias vào mỗi hàng. Sau đó, softmax chuyển 10 điểm của mỗi ảnh thành xác suất:

$$Z = XW + \mathbf{1}_B \mathbf{b}^\top, \qquad p_{ic} = \frac{\exp(z_{ic} - a_i)}{\sum_{k=0}^{9} \exp(z_{ik} - a_i)}, \quad a_i = \max_k z_{ik}. \tag{5}$$

Trừ a_i giữ nguyên xác suất và tránh tràn số khi lấy hàm mũ.

Khi huấn luyện, nhóm dùng cross-entropy (CE) trung bình trên mini-batch và phạt L2 với hệ số λ trên W:

$$\mathcal{L}(W, \mathbf{b}) = -\frac{1}{B} \sum_{i=1}^{B} \log p_{i, y_i} + \frac{\lambda}{2} \|W\|_F^2. \tag{6}$$

Để tạo ma trận E, nhóm lấy ma trận xác suất P và trừ 1 ở phần tử hàng i, cột y_i, với y_i là nhãn thật của ảnh thứ i. Với tốc độ học η, các bước tính gradient và cập nhật là

$$\nabla_W \mathcal{L} = X^\top E/B + \lambda W, \qquad \nabla_{\mathbf{b}} \mathcal{L} = \frac{1}{B} \sum_i (E_{i,:})^\top, \tag{7}$$

$$W \leftarrow W - \eta \nabla_W \mathcal{L}, \qquad \mathbf{b} \leftarrow \mathbf{b} - \eta \nabla_{\mathbf{b}} \mathcal{L}. \tag{8}$$

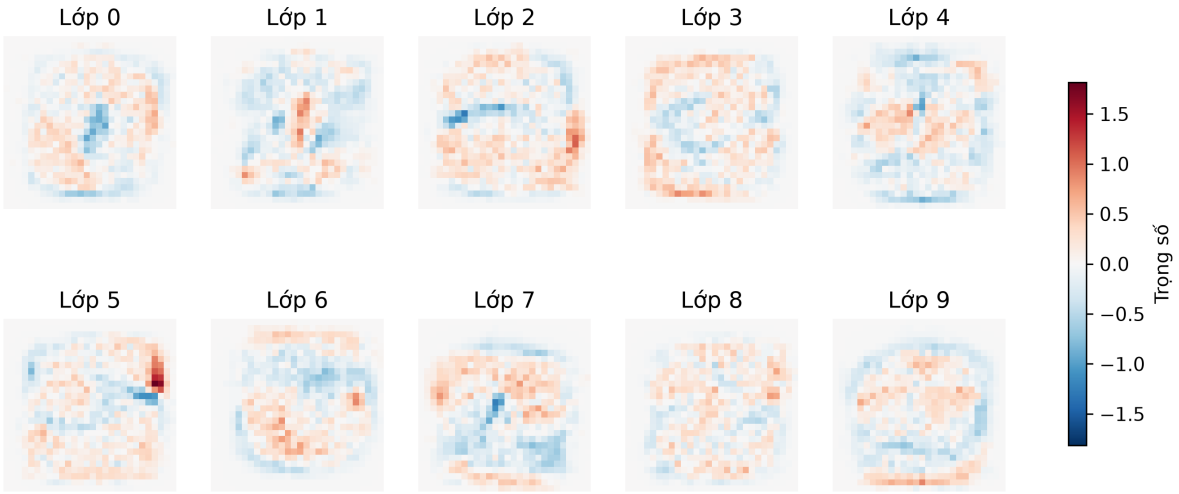
Hàm `_loss()` tính cross-entropy (CE) để theo dõi quá trình huấn luyện và chọn bộ tham số theo tập validation. Thành phần L2 chỉ xuất hiện trong gradient của W.

Mã giả 2. Softmax với mini-batch gradient descent

1. Khởi tạo $W \leftarrow 0$, $\mathbf{b} \leftarrow 0$, $\text{CE}_{\text{best}} \leftarrow +\infty$ và đặt seed.
2. Mỗi epoch, xáo trộn chỉ số train và chia thành các mini-batch.
3. Mỗi mini-batch: $Z \leftarrow XW + \mathbf{1}_B \mathbf{b}^\top$, $P \leftarrow \text{softmax}(Z)$, $E \leftarrow P$, $E_{i,y_i} \leftarrow E_{i,y_i} - 1$.
4. Cập nhật $W \leftarrow W - \eta(X^\top E/B + \lambda W)$, $\mathbf{b} \leftarrow \mathbf{b} - \frac{\eta}{B} \sum_i (E_{i,:})^\top$.
5. Ghi CE và accuracy. Nếu $\text{CE}_{\text{val}} < \text{CE}_{\text{best}} - 10^{-6}$, lưu $(W, \mathbf{b})$ và đặt $\text{CE}_{\text{best}} \leftarrow \text{CE}_{\text{val}}$.
6. Khôi phục $(W, \mathbf{b})$ đã lưu. Dự đoán $\widehat{y}_i = \arg\max_c p_{ic}$.

Mô hình được huấn luyện trong 100 epoch. Sau huấn luyện, nhóm khôi phục bộ tham số đã lưu ở epoch 58 theo CE trên tập validation. Mô hình có 784×10 trọng số và 10 hệ số bias, tổng cộng 7.850 tham số.

Mỗi cột của W chứa 784 trọng số ứng với một chữ số. Nhóm đưa từng cột về dạng 28×28 để xem vị trí pixel nào có ảnh hưởng lớn đến điểm của mỗi lớp (Hình 3).



Hình 3
Bản đồ trọng số của Softmax Regression cho 10 lớp chữ số.

Vùng đỏ biểu thị trọng số dương, vùng xanh biểu thị trọng số âm. Khi một pixel có cường độ lớn, nó làm tăng hoặc giảm logit của lớp tùy theo dấu của trọng số tương ứng. Chẳng hạn, bản đồ lớp 0 có vùng trọng số âm ở giữa và các vùng dương ở phía ngoài, phù hợp với dạng nét vòng thường thấy của chữ số này.

3.3 Mạng nhiều lớp MLP bằng NumPy

Mạng có L lớp tham số. Với mini-batch X gồm B ảnh, đặt $A^{(0)} = X$. Lan truyền thuận được tính như sau:

$$Z^{(\ell)} = A^{(\ell-1)} W^{(\ell)} + \mathbf{1}_B (\mathbf{b}^{(\ell)})^\top, \tag{9}$$

$$A^{(\ell)} = \max(0, Z^{(\ell)}), \quad 1 \le \ell < L, \qquad P = \mathrm{softmax}(Z^{(L)}). \tag{10}$$

Trong công thức (9), mỗi lớp nhận đầu ra của lớp trước, nhân với trọng số và cộng bias. Ở ba lớp ẩn, ReLU giữ các giá trị dương và đưa các giá trị còn lại về 0. Nhờ bước phi tuyến này, mạng có thể học cách kết hợp các đặc trưng từ 784 pixel qua các lớp ẩn 256, 128 và 64 nút. Lớp ra chuyển 10 điểm của mỗi ảnh thành xác suất bằng softmax rồi chọn chữ số có xác suất lớn nhất.

Sau mỗi epoch, hàm `_loss()` ghi loss trên toàn bộ tập train. Công thức dưới đây viết cho B mẫu đầu vào của hàm:

$$\mathcal{L}_{\text{ghi}} = -\frac{1}{B} \sum_i \log(p_{i,y_i} + 10^{-12}) + \frac{\lambda}{2} \sum_{\ell=1}^{L} \|W^{(\ell)}\|_F^2. \tag{11}$$

Cross-entropy giảm khi mô hình gán xác suất cao hơn cho nhãn đúng. Thành phần L2 phạt trọng số lớn ở tất cả các lớp. Số hạng 10^{-12} giữ đối số của log dương khi tính loss. Khi cập nhật theo mini-batch, mã dùng gradient của CE chuẩn cộng L2, tức $\mathcal{L}_{\text{SGD}}$ có cùng dạng với công thức (6) nhưng phạt L2 trên trọng số của tất cả các lớp. Đặt $G^{(L)} = (P - Y)/B$, với

$Y_{ic} = \mathbb{1}[c = y_i]$ là ký hiệu cho nhãn thật:

$$\nabla_{W^{(\ell)}} \mathcal{L}_{\text{SGD}} = (A^{(\ell-1)})^\top G^{(\ell)} + \lambda W^{(\ell)}, \tag{12}$$

$$\nabla_{\mathbf{b}^{(\ell)}} \mathcal{L}_{\text{SGD}} = \sum_i (G^{(\ell)}_{i,:})^\top, \tag{13}$$

$$G^{(\ell-1)} = G^{(\ell)} (W^{(\ell)})^\top \odot \mathbb{1}[A^{(\ell-1)} > 0], \quad 2 \le \ell \le L. \tag{14}$$

Tín hiệu $G^{(\ell)}$ cho biết loss thay đổi thế nào theo các giá trị $Z^{(\ell)}$. Khi truyền về lớp trước, mã nhân tín hiệu này với ma trận trọng số chuyển vị và chỉ giữ gradient tại các nút có đầu ra ReLU dương. Bước này dùng $W^{(\ell)}$ của lượt lan truyền thuận, sau đó mới cập nhật trọng số và bias.

Mã giả 3. Feedforward và backpropagation của MLP

1. Ánh xạ nhãn sang chỉ số lớp. Với mỗi ℓ, lấy $W^{(\ell)}, \mathbf{b}^{(\ell)}$ từ $\mathcal{U}(-r_\ell, r_\ell)$, với $r_\ell = \sqrt{6/(n_{\ell-1} + n_\ell)}$.
2. Mỗi epoch, xáo trộn ảnh train và chia thành các mini-batch.
3. **Lan truyền thuận.** Với mỗi mini-batch, đặt $A^{(0)} \leftarrow X$.
   Với $\ell = 1, \dots, L-1$: $Z^{(\ell)} \leftarrow A^{(\ell-1)} W^{(\ell)} + \mathbf{1}_B (\mathbf{b}^{(\ell)})^\top$, $A^{(\ell)} \leftarrow \max(0, Z^{(\ell)})$.
   Lớp ra: $Z^{(L)} \leftarrow A^{(L-1)} W^{(L)} + \mathbf{1}_B (\mathbf{b}^{(L)})^\top$, $P \leftarrow \text{softmax}(Z^{(L)})$.
4. **Lan truyền ngược.** Đặt $G^{(L)} \leftarrow P$, trừ 1 tại mỗi phần tử $G^{(L)}_{i,y_i}$, rồi chia $G^{(L)}$ cho B.
   Với $\ell = L, \dots, 1$, thực hiện theo thứ tự:
   $dW^{(\ell)} \leftarrow (A^{(\ell-1)})^\top G^{(\ell)} + \lambda W^{(\ell)}$, $d\mathbf{b}^{(\ell)} \leftarrow \sum_i (G^{(\ell)}_{i,:})^\top$.

   Nếu $\ell > 1$: $G^{(\ell-1)} \leftarrow G^{(\ell)} (W^{(\ell)})^\top \odot \mathbb{1}[A^{(\ell-1)} > 0]$.
   $W^{(\ell)} \leftarrow W^{(\ell)} - \eta\, dW^{(\ell)}$, $\mathbf{b}^{(\ell)} \leftarrow \mathbf{b}^{(\ell)} - \eta\, d\mathbf{b}^{(\ell)}$.
5. Ghi loss train sau mỗi epoch. Dùng tham số cuối để dự đoán $\hat{y}_i$ từ $\arg\max_c p_{ic}$.

Mô hình được huấn luyện trong 50 epoch bằng SGD. Tổng số tham số là $\sum_{\ell=1}^{L} (n_{\ell-1} + 1) n_\ell = 242.762$.

3.4 Stacking với AdaBoost làm meta-learner

Lựa chọn mô hình cơ sở và voting

Notebook 05 khảo sát sáu mô hình: SVM RBF, Softmax của scikit-learn, Random Forest, KNN, Extra Trees và MLP256_128. Các bộ ba được xếp hạng theo accuracy của soft voting trên validation, sau đó lần lượt theo F1 macro cao hơn, tỷ lệ cùng sai trung bình giữa các cặp thấp hơn và tên bộ ba. Nhóm chọn SVM, KNN và MLP256_128. Soft voting lấy trung bình ba xác suất cho từng lớp:

$$\overline{p}_c(\mathbf{x}) = \frac{1}{3} \sum_{j=1}^{3} p_c^{(j)}(\mathbf{x}), \qquad \hat{y}_{\text{vote}} = \arg\max_c \overline{p}_c(\mathbf{x}). \tag{15}$$

Weighted soft voting dùng trung bình có trọng số $\sum_{j=1}^{3} \beta_j p_c^{(j)}(\mathbf{x})$, với $\beta_j \ge 0$ và $\sum_j \beta_j = 1$. Trọng số được chọn trên validation đều bằng $1/3$, nên hai cách voting cho cùng kết quả trong thí nghiệm này.

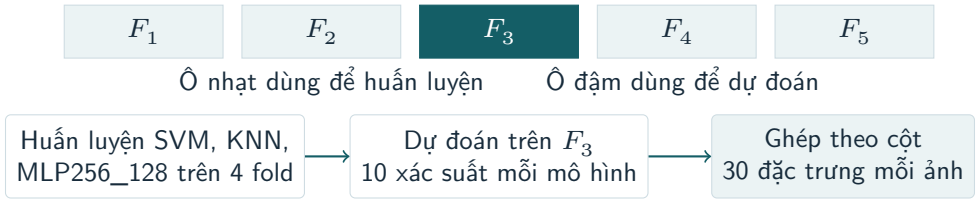
Đặc trưng OOF và học tầng meta

Để tầng meta học từ dự đoán trên những ảnh mà mô hình cơ sở chưa dùng để huấn luyện, nhóm chia 50.000 ảnh train thành 5 fold phân tầng. Với mỗi mô hình cơ sở, nhóm huấn luyện trên 4 fold và dự đoán xác suất cho fold còn lại. Mỗi ảnh train có một vector xác suất

ngoài fold (out-of-fold, OOF) từ từng mô hình. Ghép ba vector theo thứ tự SVM, KNN và MLP256_128:

$$\mathbf{z}_i = \left[p_0^{(1)}, \dots, p_9^{(1)},\ p_0^{(2)}, \dots, p_9^{(2)},\ p_0^{(3)}, \dots, p_9^{(3)}\right], \quad Z_{\text{OOF}} \in \mathbb{R}^{50\,000\times 30}. \tag{16}$$

Hình 4 minh họa lượt giữ lại fold F_3. Lặp lại với từng fold để mỗi ảnh nhận đúng một bộ xác suất từ mỗi mô hình. Các vector được ghép theo cột rồi ghi về đúng hàng của ảnh trong Z_{OOF}.



Hình 4
Tạo đặc trưng OOF cho một fold trong quy trình stacking.

Sau khi tạo Z_{OOF}, nhóm huấn luyện lại ba mô hình cơ sở trên toàn bộ tập train để tạo đặc trưng khi đánh giá. Vì mô hình trong mỗi fold học từ 40.000 ảnh, còn mô hình huấn luyện lại học từ 50.000 ảnh, phân bố xác suất đầu ra có thể thay đổi giữa hai bước.

Tầng meta dùng các cây quyết định làm bộ phân lớp yếu và nhận 30 xác suất trong mỗi hàng của Z_{OOF}. Với SAMME, gọi $w_i^{(t)}$ là trọng số mẫu đã chuẩn hóa ở vòng t, tức $\sum_i w_i^{(t)} = 1$. Sai số và hệ số của cây h_t trong một vòng cập nhật thông thường được tính theo SAMME [5].

$$\epsilon_t = \sum_i w_i^{(t)} \mathbb{1}[h_t(\mathbf{z}_i) \neq y_i], \quad \alpha_t = \eta \left[\log \frac{1-\epsilon_t}{\epsilon_t} + \log(K-1)\right], \quad K = 10. \tag{17}$$

Với $0 < \epsilon_t < 1 - 1/K$, SAMME nhân trọng số của mẫu bị phân loại sai với $\exp(\alpha_t)$ rồi chuẩn hóa lại. Nhóm tự triển khai việc chia fold, tạo đặc trưng OOF và ghép xác suất. Thuật toán SAMME ở tầng meta do `AdaBoostClassifier` của scikit-learn thực hiện.

Mã giả 4. Quy trình stacking của nhóm

1. Nạp ba cấu hình từ notebook 05. Chia train thành 5 fold phân tầng.
2. Với mô hình j và fold F_k: huấn luyện trên phần train ngoài F_k rồi gán $Z_{\text{OOF}}[F_k, 10(j-1) : 10j] \leftarrow M_{j,k}.\text{predict_proba}(X[F_k])$, với $j = 1, 2, 3$.
3. Huấn luyện lại M_1, M_2, M_3 trên toàn bộ train. Với mỗi tập s, ghép $Z_s \leftarrow [P_s^{(1)} \mid P_s^{(2)} \mid P_s^{(3)}]$ từ xác suất của ba mô hình này.
4. Khảo sát tốc độ học, độ sâu và số cây. Huấn luyện AdaBoost trên $(Z_{\text{OOF}}, y_{\text{train}})$ và xếp hạng theo accuracy, rồi F1 macro trên validation. Nếu bằng điểm, ưu tiên độ sâu, số cây và tốc độ học nhỏ hơn theo thứ tự.
5. Huấn luyện lại tầng meta với cấu hình đã chọn. Với mỗi tập s, lấy $\hat{y}_s \leftarrow \text{meta.predict}(Z_s)$ và so sánh với mô hình đơn và hai cách voting.

Cấu hình cuối dùng 100 cây, độ sâu tối đa 2 và tốc độ học 0,2. Notebook lấy arg max xác suất cho các mô hình cơ sở và hai cách voting. Với stacking, nhãn được lấy trực tiếp từ `meta.predict()`.

4 Thiết kế thực nghiệm

4.1 Giao thức đánh giá

Các mô hình sử dụng giao thức ở Bảng 1. Sau khi chọn cấu hình, mô hình cuối được đánh giá trên cả ba tập bằng cùng quy tắc dự đoán. Điểm validation phản ánh chất lượng trên tập đã dùng để chọn cấu hình. Điểm test phản ánh chất lượng trên tập kiểm thử gốc sau khi chốt cấu hình. Điểm train của stacking được tính bằng các base learner đã refit trên toàn bộ train, còn đặc trưng dùng để học tầng meta đến từ OOF.

Bảng 2
Cấu hình cuối theo các file `config.json` của notebook 02–06.

Phương pháp	Cấu hình cuối	Tiêu chí hoặc quy tắc chọn
Gaussian NB	$\varepsilon = 10^{-3}$	Giữ cấu hình cuối của notebook 02, có đối chiếu khảo sát validation.
Softmax	$\eta = 0{,}5$, $\lambda = 2 \times 10^{-5}$, batch 256, 100 epoch	F1 macro validation, CE phá hòa. Checkpoint epoch 58 theo CE.
MLP	Lớp ẩn 256–128–64, $\eta = 0{,}1$, $\lambda = 10^{-5}$, batch 32, 50 epoch	Kết hợp các giá trị đã chọn qua từng lượt khảo sát validation.
Stacking AdaBoost	SVM + KNN + MLP256_128, OOF 5 fold, 100 cây sâu 2, $\eta = 0{,}2$	Bộ ba theo soft voting validation. Tầng meta theo accuracy, F1 macro phá hòa.

Bảng 2 liệt kê cấu hình đã dùng để tạo kết quả chính. Với Softmax, notebook khảo sát learning rate trước, sau đó khảo sát L2 với learning rate đã chọn. Với MLP, các lượt thay đổi learning rate, kiến trúc, epoch, L2 và batch size được thực hiện riêng. Giá trị tốt nhất trong từng lượt chịu ảnh hưởng của các tham số giữ cố định. Vì thế, tổ hợp cuối là kết quả của quy trình chọn tuần tự, chưa bao phủ toàn bộ tích Descartes của các giá trị khảo sát.

Nhóm khảo sát lần lượt tốc độ học, độ sâu tối đa và số cây của AdaBoost ở tầng meta. Mỗi lượt thay đổi một thông số so với cấu hình mốc gồm tốc độ học 0,5, độ sâu tối đa 2 và 100 cây. Khi khảo sát số cây, nhóm huấn luyện một mô hình với tối đa 300 cây rồi dùng `staged_predict()` để đánh giá tại các mốc khác nhau. Cấu hình cuối được chọn dựa trên kết quả validation của các cấu hình đã thử.

4.2 Độ đo phân lớp

Trong ma trận nhầm lẫn, C_{ij} là số ảnh có nhãn thật i được dự đoán thành j. Với lớp c, số dự đoán đúng là $TP_c = C_{cc}$, số ảnh bị gán nhầm vào lớp là $FP_c = \sum_i C_{ic} - C_{cc}$ và số ảnh thuộc lớp bị bỏ sót là $FN_c = \sum_j C_{cj} - C_{cc}$. Các độ đo được tính như sau:

$$\mathrm{Acc} = \frac{\sum_c C_{cc}}{\sum_{i,j} C_{ij}}, \qquad P_c = \frac{TP_c}{TP_c + FP_c}, \qquad R_c = \frac{TP_c}{TP_c + FN_c}, \qquad F1_c = \frac{2P_cR_c}{P_c + R_c}. \tag{18}$$

Accuracy là tỷ lệ ảnh được dự đoán đúng trên toàn tập. Precision cho biết trong số ảnh được dự đoán là lớp c, bao nhiêu ảnh thực sự thuộc lớp đó. Recall cho biết trong số ảnh thực sự thuộc lớp c, bao nhiêu ảnh được nhận dạng đúng. F1 là trung bình điều hòa của precision và recall. Khi mẫu số bằng 0, độ đo tương ứng được đặt bằng 0.

Các giá trị macro là trung bình của từng độ đo trên 10 lớp. Với $M \in \{P, R, F1\}$, ta có $M_{\mathrm{macro}} = \frac{1}{10}\sum_{c=0}^{9} M_c$. Ma trận nhầm lẫn chuẩn hóa theo hàng có phần tử $\widetilde{C}_{ij} = C_{ij} / \sum_k C_{ik}$, nên giá trị trên đường chéo là recall của từng lớp.

4.3 Đo thời gian và đối chiếu thư viện

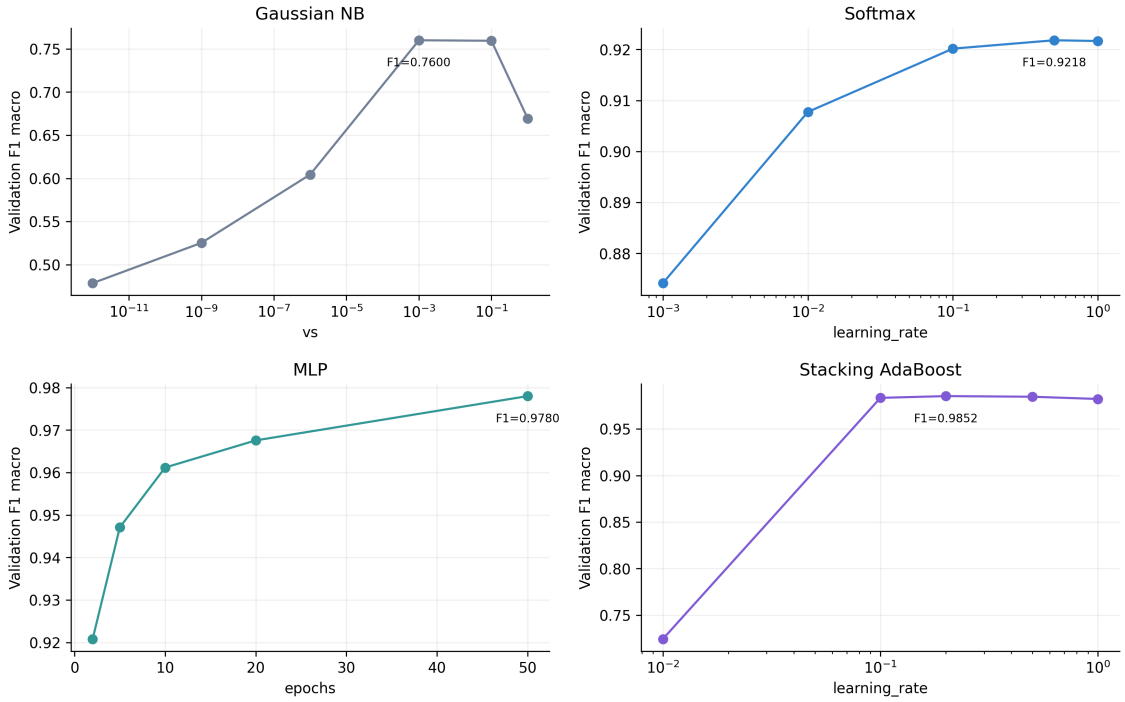
Báo cáo ghi thời gian huấn luyện cấu hình cuối và tổng thời gian dự đoán 10.000 ảnh test. Với stacking, thời gian huấn luyện là tổng thời gian tạo đặc trưng OOF, huấn luyện lại các mô hình cơ sở trên toàn bộ tập train và huấn luyện AdaBoost ở tầng meta. Thời gian chọn bộ ba và khảo sát siêu tham số tầng meta được ghi riêng. Thời gian dự đoán gồm tính xác suất từ các mô hình cơ sở và dự đoán bằng AdaBoost. Nếu quy đổi sang thời gian trung bình mỗi ảnh, giá trị này mô tả tốc độ xử lý trên cả tập test.

Số liệu thời gian lấy từ các lượt chạy đã lưu. Hai thí nghiệm CNN chạy trên CPU. Các phép đo được thực hiện trong điều kiện chạy của từng notebook, nên thời gian được so sánh trong phạm vi các lượt chạy đó. Phần đối chiếu với scikit-learn được trình bày riêng để phân biệt kết quả của cấu hình cuối với phép kiểm tra khi thiết lập tham số tương ứng.

5 Kết quả thực nghiệm

5.1 Ảnh hưởng của siêu tham số trên validation

Hình 5 cho thấy F1 macro trên tập validation khi thay đổi một siêu tham số của mỗi phương pháp. Các thông số còn lại được giữ cố định trong từng lượt khảo sát.



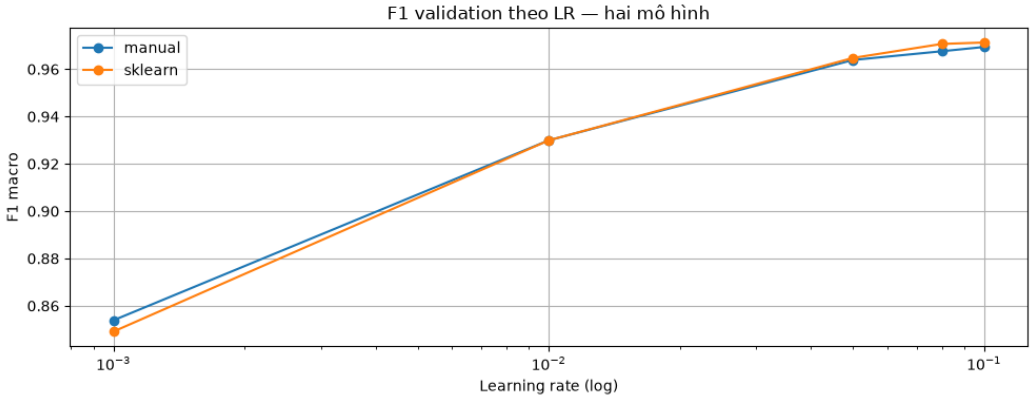
Hình 5
F1 macro trên validation theo hệ số làm trơn của Gaussian NB, tốc độ học của Softmax, số epoch của MLP và tốc độ học của AdaBoost ở tầng meta.

Gaussian NB. Khi ε tăng từ 10^{-12} lên 10^{-3}, F1 macro tăng từ 0,4786 lên 0,7600. Hai mức 10^{-3} và 10^{-1} cho kết quả gần nhau, còn ở $\varepsilon = 1$, điểm giảm xuống 0,6694. Trong công thức (4), phương sai nằm ở mẫu số của hạng bình phương độ lệch. Cộng ε giúp các pixel có phương sai rất nhỏ bớt chi phối điểm log-joint. Khi hệ số quá lớn, phương sai dùng trong mô hình ít phản ánh sự khác biệt giữa các lớp.

Softmax. Trong lượt khảo sát 100 epoch, F1 macro tăng từ 0,8742 ở tốc độ học 0,001 lên

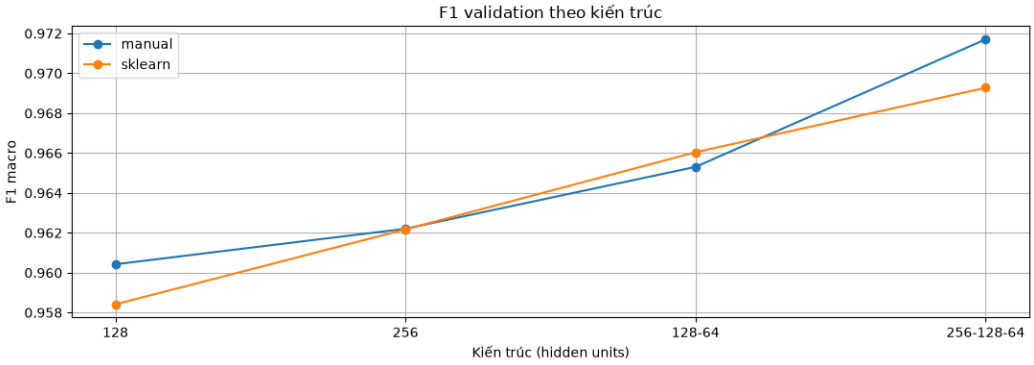
0,9218 ở mức 0,5. Khi tăng tiếp lên 1,0, điểm giảm nhẹ còn 0,9216. Ở lượt khảo sát L2 với tốc độ học 0,5, mức $\lambda = 2 \times 10^{-5}$ đạt 0,9218, nhỉnh hơn mức $\lambda = 0$ (0,9213).

MLP. Các lượt khảo sát trong notebook thay đổi từng tham số riêng, dùng cấu hình mốc gồm kiến trúc 256–128, tốc độ học 0,08, 20 epoch, batch size 256 và L2 bằng 10^{-4}. Khi tăng tốc độ học qua các mức 0,001; 0,01; 0,05; 0,08 và 0,1, F1 macro validation của MLP tự viết tăng lần lượt từ 0,8539 lên 0,9299; 0,9639; 0,9676 và 0,9694. MLP scikit-learn cũng đạt cao nhất trong lượt này ở tốc độ học 0,1 (0,9713; so với 0,8491 ở 0,001). Hình 6 minh họa kết quả khảo sát.



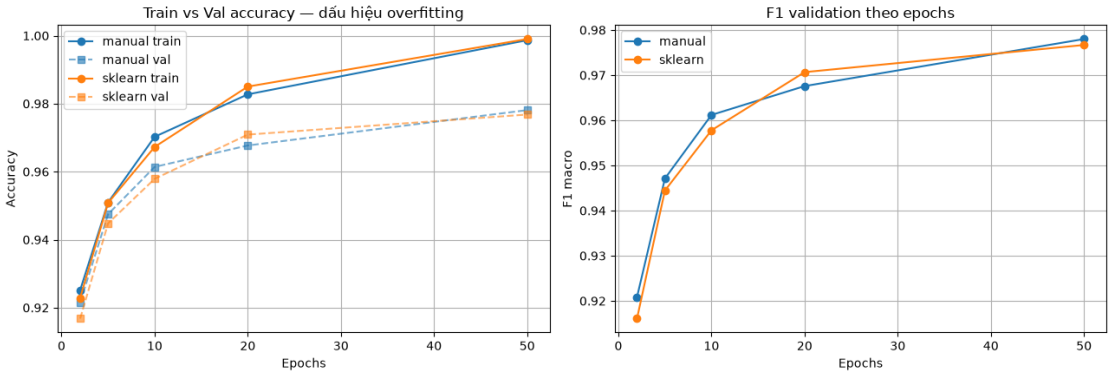
Hình 6
F1 macro validation theo tốc độ học của MLP tự viết và scikit-learn.

Trong khảo sát kiến trúc, F1 macro của MLP tự viết tăng từ 0,9604 với một lớp ẩn 128 neuron, lên 0,9622 với 256 neuron, 0,9653 với hai lớp 128–64 và 0,9717 với ba lớp 256–128–64. Scikit-learn đạt cao nhất 0,9693 với kiến trúc 256–128–64. Kết quả được trình bày ở Hình 7.



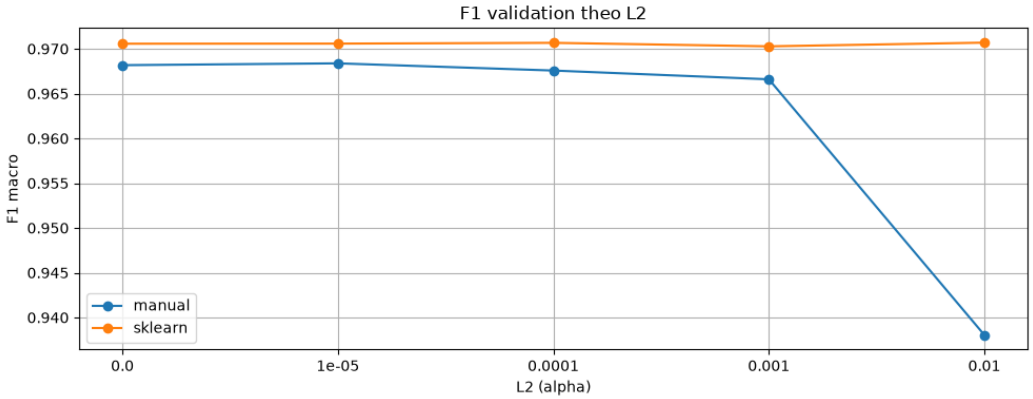
Hình 7
F1 macro validation theo kiến trúc các lớp ẩn.

Khi khảo sát số epoch, F1 macro của MLP tự viết tăng từ 0,9208 ở 2 epoch lên 0,9471, 0,9612, 0,9676 và 0,9780 tương ứng tại 5, 10, 20 và 50 epoch. Accuracy train ở 50 epoch là 99,87%, trong khi accuracy validation là 97,81%; khoảng cách này cho thấy mức khớp train cao hơn validation (Hình 8).



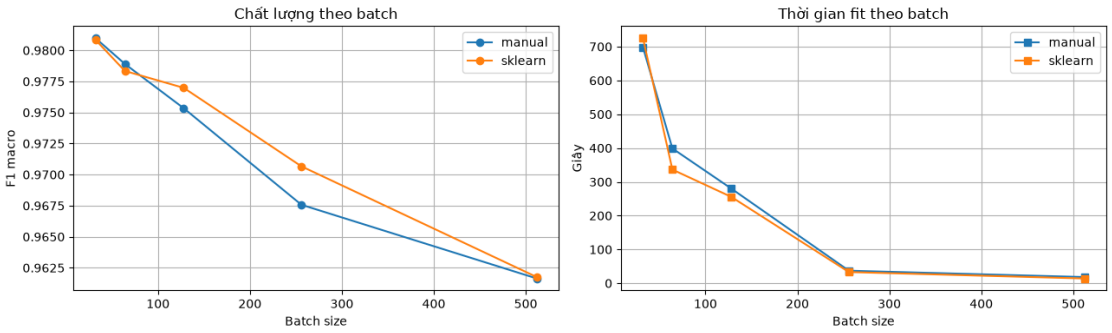
Hình 8
Accuracy train/validation và F1 macro validation theo số epoch.

Với L2, MLP tự viết đạt F1 macro 0,9682 khi không regularization và cao nhất 0,9684 tại 10^{-5}; khi tăng lên 10^{-2}, điểm giảm còn 0,9380. Kết quả scikit-learn ít thay đổi giữa các mức L2 đã thử, với F1 macro trong khoảng 0,9703–0,9707 (Hình 9).



Hình 9
F1 macro validation theo hệ số L2.

Batch size nhỏ hơn cho điểm validation cao hơn trong lượt khảo sát: MLP tự viết đạt 0,9810 ở batch 32, rồi giảm qua 0,9789, 0,9753, 0,9676 xuống 0,9616 ở batch 512. Xu hướng tương tự xuất hiện ở scikit-learn, từ 0,9808 xuống 0,9618 (Hình 10). Từ các lượt khảo sát riêng, cấu hình cuối của MLP tự viết kết hợp kiến trúc 256–128–64, tốc độ học 0,1, 50 epoch, batch 32 và L2 bằng 10^{-5}; tổ hợp này được đánh giá riêng trong kết quả cuối, không phải kết quả của một lượt tìm kiếm toàn bộ tổ hợp tham số.



Hình 10
F1 macro validation và thời gian huấn luyện theo batch size.

AdaBoost ở tầng meta. Với độ sâu tối đa 2 và 100 cây, F1 macro tăng từ 0,7245 ở tốc độ học 0,01 lên 0,9852 ở mức 0,2. Khi tăng tốc độ học lên 1,0, điểm giảm còn 0,9821. Các mức từ 0,1 đến 0,5 cho kết quả khá gần nhau, trong đó mức 0,2 đạt accuracy và F1 macro cao nhất trong lượt khảo sát tốc độ học.

5.2 Kết quả cuối trên train, validation và test

Bảng 3
Các độ đo trên train và validation (%). P, R và F1 là giá trị macro.

Phương pháp	Tập	Acc.	P	R	F1
Gaussian NB	Train	76,83	79,23	76,34	76,41
	Validation	76,35	79,09	75,87	76,00
Softmax	Train	93,33	93,25	93,23	93,23
	Validation	92,28	92,20	92,19	92,18
MLP	Train	100,00	100,00	100,00	100,00
	Validation	98,30	98,29	98,28	98,29
Stacking AdaBoost	Train	99,998	99,998	99,998	99,998
	Validation	98,53	98,52	98,53	98,52

Bảng 4
Kết quả trên 10.000 ảnh test. Bốn độ đo được biểu diễn bằng phần trăm.

Phương pháp	Acc.	P	R	F1	Số ảnh sai
Gaussian NB	77,53	80,19	77,07	77,16	2247
Softmax	92,53	92,44	92,41	92,41	747
MLP	98,19	98,19	98,17	98,18	181
Stacking AdaBoost	98,54	98,54	98,53	98,54	146

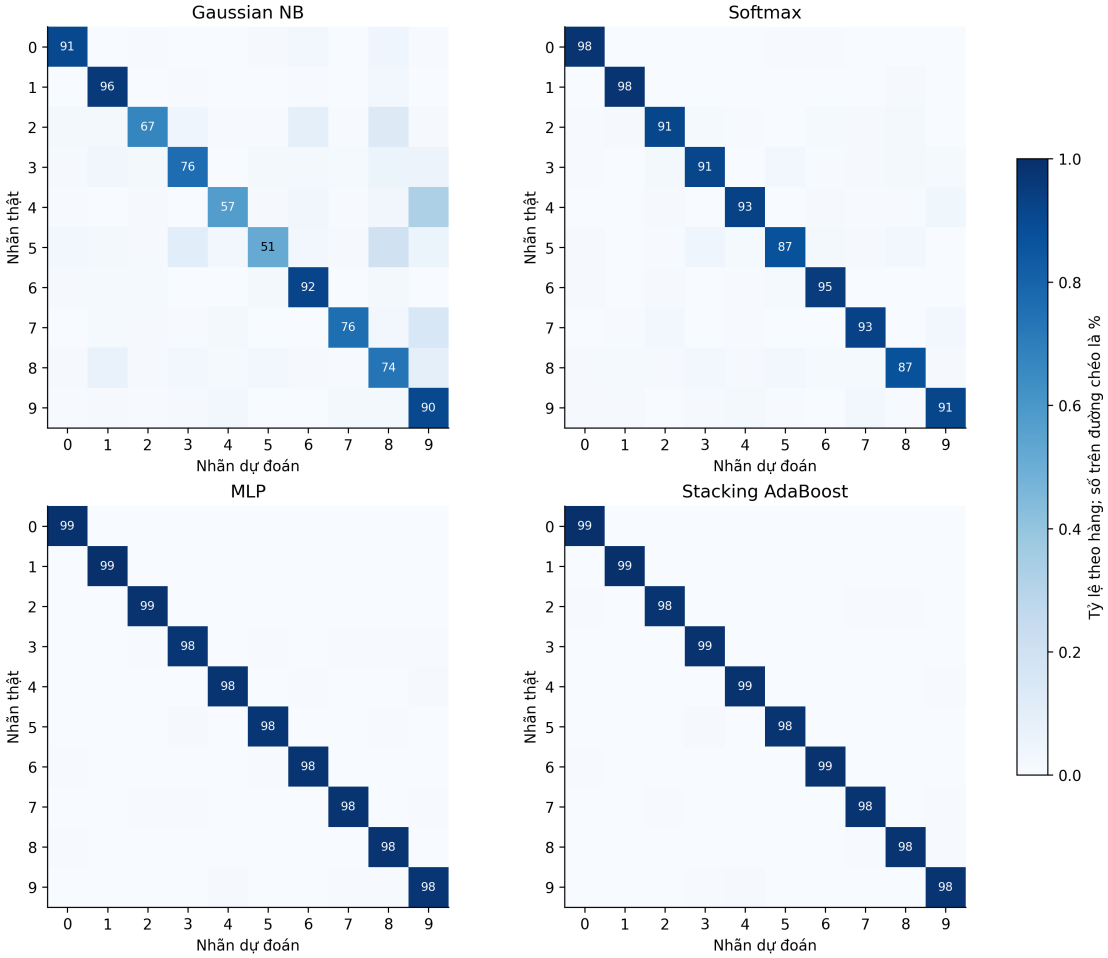
Trên 10.000 ảnh test (Bảng 4), Softmax dự đoán sai ít hơn Gaussian NB 1.500 ảnh, tương ứng mức tăng accuracy 15,00 điểm phần trăm. MLP giảm thêm 566 lỗi so với Softmax, còn Stacking AdaBoost giảm thêm 35 lỗi so với MLP. Stacking đạt accuracy và F1 macro cao nhất, cùng làm tròn ở mức 98,54%.

Ở Bảng 3, MLP đạt 100,00% trên train và 98,30% trên validation, chênh 1,70 điểm phần trăm. Gaussian NB có mức chênh nhỏ hơn (0,48 điểm phần trăm), với accuracy lần lượt là 76,83% và 76,35% trên hai tập.

Accuracy trên validation và test chênh tối đa 0,25 điểm phần trăm với Softmax, MLP và Stacking AdaBoost. Gaussian NB có chênh lệch lớn nhất giữa hai tập, từ 76,35% trên validation lên 77,53% trên test.

5.3 Lỗi theo lớp và đối chiếu dự đoán từng ảnh

Để xem mỗi mô hình thường nhầm chữ số nào, nhóm đặt bốn ma trận nhầm lẫn trên tập test cạnh nhau. Mỗi hàng ứng với một nhãn thật và được chuẩn hóa theo số ảnh của lớp đó. Ô trên đường chéo là recall, còn các ô ngoài đường chéo cho biết ảnh của lớp đó bị gán sang những nhãn nào.



Hình 11

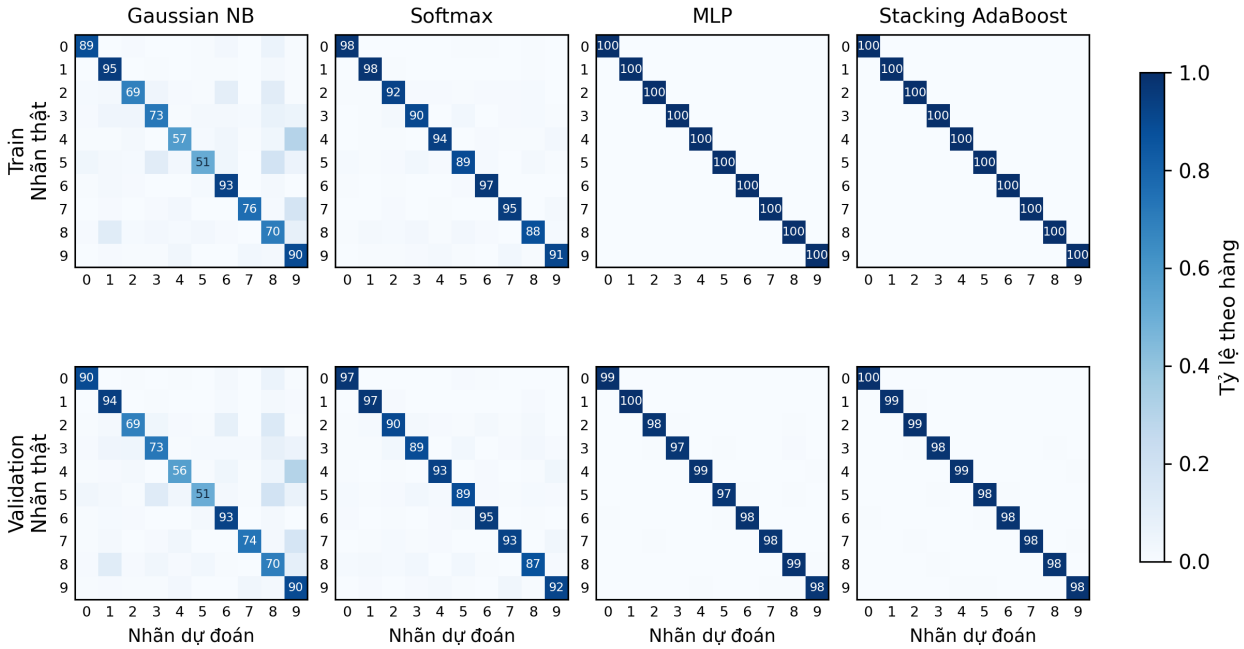
Ma trận nhầm lẫn chuẩn hóa theo hàng trên tập test. Số trên đường chéo là recall làm tròn theo phần trăm.

Ở Hình 11, hướng nhầm từ 4 sang 9 nổi bật ở Gaussian NB với 319 ảnh. Số ảnh nhầm theo hướng này giảm còn 36 ở Softmax, 10 ở MLP và 6 ở Stacking AdaBoost. Gaussian NB còn nhầm 176 ảnh số 5 thành số 8. Với Softmax, hướng nhầm nhiều nhất là từ 5 sang 3 (39 ảnh), còn lớp có recall thấp nhất là số 8 (86,76%). Recall tính trên tất cả ảnh thuộc một lớp, nên bao gồm cả lỗi phân tán sang nhiều nhãn.

Trong phần khảo sát dữ liệu, chữ số 4 và 9 có hai ảnh trung bình gần nhau nhất theo khoảng cách Euclid. Cặp này cũng xuất hiện trong các lỗi trên test, chẳng hạn hướng $4 \to 9$ ở Gaussian NB, Softmax và MLP. Để biết nét nào dễ gây nhầm, cần xem các ảnh bị phân loại sai cụ thể.

Khi đối chiếu trên cùng tập test, MLP dự đoán đúng 607 ảnh mà Softmax dự đoán sai. Ngược lại, có 41 ảnh Softmax dự đoán đúng nhưng MLP dự đoán sai. Tính cả hai chiều, MLP giảm $607 - 41 = 566$ lỗi so với Softmax.

Hình 12 cho thấy lỗi theo lớp trên train và validation. Đường chéo của MLP trên train đều đạt 100%, còn trên validation mô hình vẫn nhầm một số ảnh. Với Gaussian NB, lớp 5 có recall thấp ở cả hai tập. Điểm train của stacking được đo sau khi ba mô hình cơ sở đã được huấn luyện lại trên toàn bộ tập train (mục 3.4).



Hình 12

Ma trận nhầm lẫn chuẩn hóa theo hàng trên train và validation. Đường chéo ghi recall theo phần trăm.

5.4 Đóng góp của voting và tầng AdaBoost

Trên tập validation, SVM đạt accuracy 98,33%, cao nhất trong các mô hình đơn được khảo sát ở notebook 05. Soft voting giữa SVM và MLP256_128 đạt 98,38%. Khi thêm KNN, accuracy của bộ ba đạt 98,52%, tăng 0,14 điểm phần trăm so với cặp và 0,19 điểm phần trăm so với SVM.

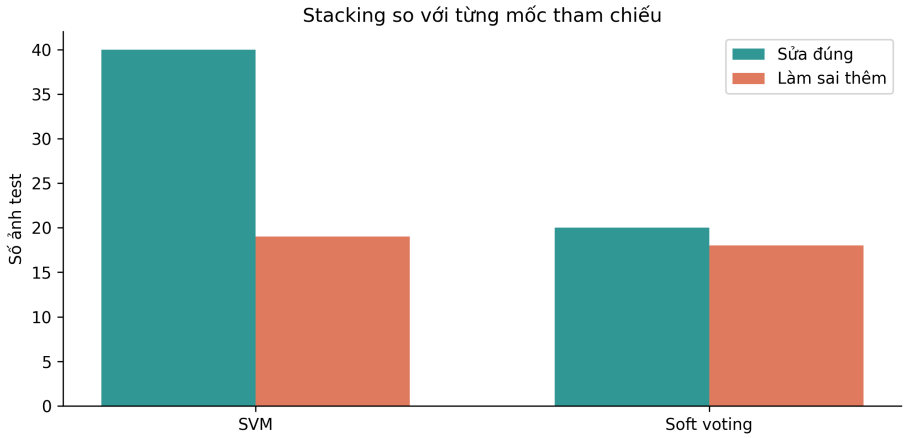
Bảng 5

Accuracy (%) của base learners, voting và stacking trong notebook 06.

Phương án	Validation	Test
SVM	98,33	98,33
KNN	97,20	97,00
MLP256_128	98,19	98,26
Soft voting	98,52	98,52
Weighted soft voting	98,52	98,52
Stacking AdaBoost	98,53	98,54

Trên tập test (Bảng 5), soft voting đạt 98,52%, cao hơn SVM 0,19 điểm phần trăm. Ba trọng số được chọn trên validation đều bằng 1/3, nên weighted soft voting cho cùng kết quả. Soft voting gộp ba xác suất của từng lớp bằng trung bình đều. Stacking đưa cả 30 xác suất vào AdaBoost để học cách kết hợp từ dữ liệu OOF. Trên test, stacking đạt 98,54%, tăng thêm 0,02 điểm phần trăm so với soft voting.

Hình 13 cho thấy so với SVM, stacking dự đoán đúng thêm 40 ảnh và sai thêm 19 ảnh, tức giảm ròng 21 lỗi. So với soft voting, stacking dự đoán đúng thêm 20 ảnh và sai thêm 18 ảnh. Hai phương án khác nhau về kết quả đúng hoặc sai trên 38 ảnh, với mức giảm ròng 2 lỗi nghiêng về stacking.



Hình 13
Số ảnh stacking dự đoán đúng thêm và sai thêm so với SVM và soft voting trên tập test.

5.5 Đối chiếu scikit-learn

Bảng 6
Đối chiếu test accuracy (%) và F1 macro của các cấu hình cuối.

Mô hình	Acc. NumPy	Acc. sklearn	F1 NumPy	F1 sklearn
Gaussian NB	77,53	81,47	0,7716	0,8131
Softmax	92,53	92,41	0,9241	0,9230
MLP	98,19	98,41	0,9818	0,9840

Gaussian NB. Khi đặt hai bản dùng cùng lượng cộng vào phương sai ε, dự đoán trên validation trùng nhau ở cả ba mức đã kiểm tra. Trong Bảng 6, hai cấu hình cuối dùng cách đặt hệ số khác nhau. Bản NumPy cộng trực tiếp $\varepsilon = 10^{-3}$, còn scikit-learn dùng `var_smoothing`= 0,1 để tính lượng cộng theo phương sai lớn nhất trên tập train. Trên test, bản scikit-learn cao hơn 3,94 điểm phần trăm với hai cấu hình này.

Softmax. Kiểm tra bằng sai phân hữu hạn cho sai số tuyệt đối lớn nhất là $4{,}78 \times 10^{-11}$ với trọng số và $2{,}96 \times 10^{-11}$ với bias. Trong thí nghiệm cuối, bản NumPy dùng mini-batch gradient descent và chọn bộ tham số theo cross-entropy trên validation. Bản scikit-learn dùng L-BFGS. Accuracy trên test của bản NumPy cao hơn 0,12 điểm phần trăm.

MLP. Hai bản có cùng kiến trúc lớp ẩn 256–128–64, nhưng dùng thiết lập L2 và cách khởi tạo khác nhau. Trên test, bản scikit-learn đạt 98,41%, cao hơn bản NumPy 0,22 điểm phần trăm.

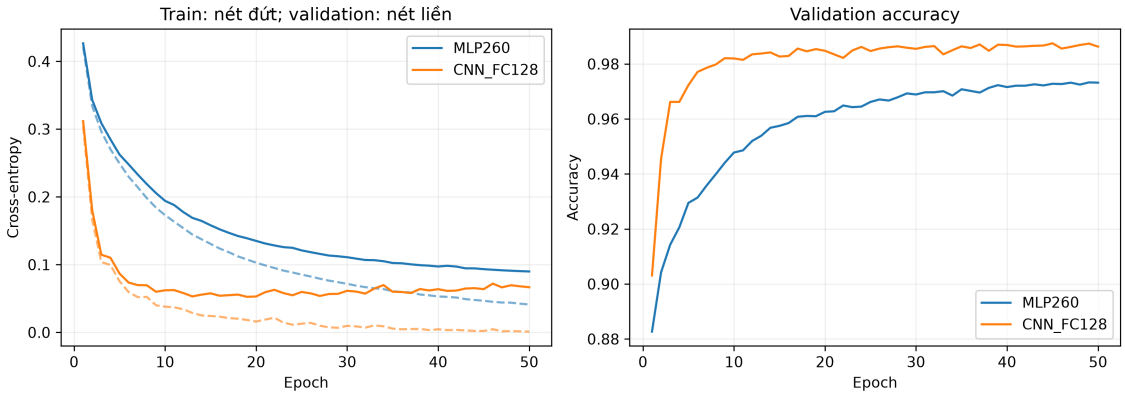
5.6 So sánh CNN và MLP có số tham số gần nhau

Trong notebook 08, nhóm so sánh hai kiến trúc có số tham số gần nhau. MLP260 nhận ảnh đã làm phẳng và dùng một lớp ẩn 260 neuron. CNN_FC128 xử lý ảnh qua hai lớp tích chập 3×3 với 16 và 32 kênh. Mỗi lớp tích chập đi kèm ReLU và max-pooling 2×2. Đầu ra $32 \times 7 \times 7$ được làm phẳng rồi đưa qua lớp ẩn 128 neuron có ReLU và lớp ra 10 logit. CNN_FC128 có 206.922 tham số, nhiều hơn 212 so với 206.710 tham số của MLP260.

Cả hai mô hình dùng cùng cách chia dữ liệu và được huấn luyện trên CPU trong 50 epoch bằng SGD với momentum 0,9, tốc độ học 0,01 và batch size 256. Nhóm chọn checkpoint có cross-entropy thấp nhất trên validation. Checkpoint của MLP260 ở epoch 50, còn checkpoint của CNN_FC128 ở epoch 19. Thời gian trong bảng được tính trên cả 50 epoch.

Bảng 7
So sánh trên test trong thí nghiệm 50 epoch của notebook 08.

Mô hình	Acc. (%)	F1 macro	Ảnh sai	Epoch chọn	Fit (s)
MLP260	97,57	0,9755	243	50	55,81
CNN_FC128	98,70	0,9870	130	19	332,49



Hình 14
Cross-entropy và accuracy theo epoch của MLP260 và CNN_FC128.

Trên tập test, CNN_FC128 đạt accuracy 98,70%, cao hơn MLP260 1,13 điểm phần trăm và dự đoán sai ít hơn 113 ảnh. F1 macro tăng từ 0,9755 lên 0,9870. Trong lượt chạy này, CNN_FC128 mất 332,49 giây để huấn luyện, còn MLP260 mất 55,81 giây. Hai kiến trúc có số tham số gần nhau nhưng thời gian huấn luyện chênh lệch rõ rệt.

5.7 Augmentation và độ bền khi ảnh bị biến đổi

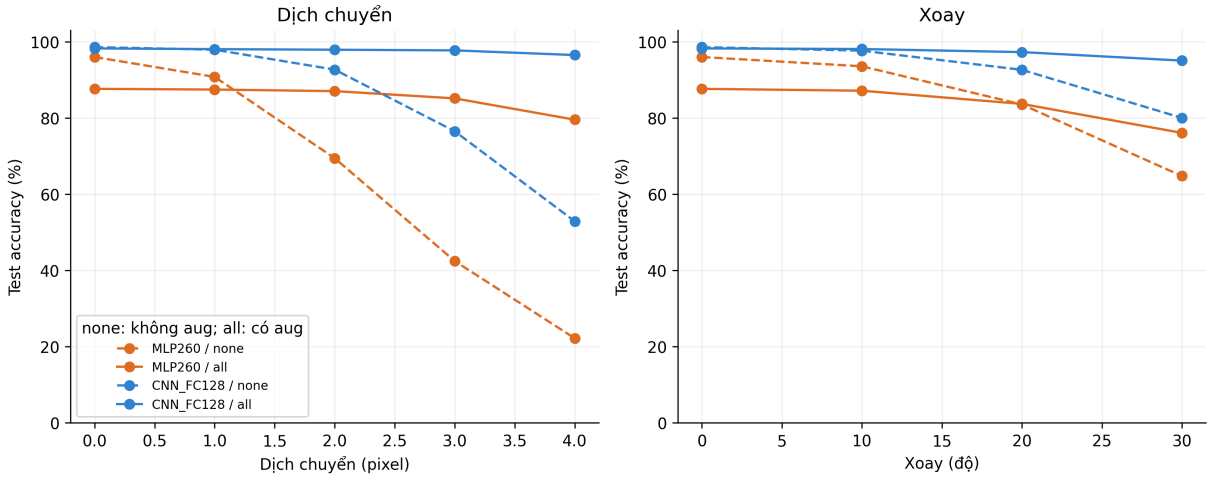
Trong notebook 09, mỗi kiến trúc được huấn luyện 15 epoch ở hai điều kiện. Điều kiện *none* dùng ảnh gốc. Điều kiện *all* tăng cường dữ liệu train bằng cách xoay trong phạm vi $\pm 30^\circ$, dịch tối đa 4 pixel và thêm nhiễu Gaussian với độ lệch chuẩn 0,1. Giá trị pixel sau biến đổi được giới hạn trong $[0, 1]$. Checkpoint được chọn theo cross-entropy thấp nhất trên tập validation gốc.

Bảng 8
Clean test trong thí nghiệm augmentation 15 epoch của notebook 09.

Mô hình	Aug.	Acc. (%)	F1 macro	Ảnh sai	Epoch chọn
CNN_FC128	Có	98,24	0,9823	176	14
CNN_FC128	Không	98,57	0,9856	143	13
MLP260	Có	87,63	0,8733	1237	12
MLP260	Không	95,98	0,9594	402	15

Trên tập test gốc (Bảng 8), khi dùng augmentation, accuracy của CNN giảm từ 98,57% xuống 98,24%, tương ứng thêm 33 ảnh sai. Với MLP260, accuracy giảm từ 95,98% xuống 87,63%, tương ứng thêm 835 ảnh sai.

Hình 15 so sánh hai chế độ huấn luyện trên cùng tập ảnh test ở từng mức biến đổi. Khi dịch 4 pixel, CNN đạt 96,50% khi có augmentation và 52,88% khi không có augmentation. Các giá trị tương ứng của MLP260 là 79,54% và 22,18%. Khi xoay 30°, CNN đạt 95,04% và 80,00%, còn MLP260 đạt 76,07% và 64,75%, theo cùng thứ tự. So với ảnh test gốc, ảnh xoay 30° làm



Hình 15

Accuracy trên tập test theo mức dịch và góc xoay. Mức 0 tương ứng với ảnh gốc.

accuracy của cả bốn cấu hình giảm.

Ở mức dịch 1 pixel, MLP260 có augmentation đạt 87,44%, thấp hơn mức 90,76% khi huấn luyện bằng ảnh gốc. Trong lượt chạy này, augmentation giúp cả hai mô hình nhận dạng tốt hơn ở các mức dịch và xoay lớn, đồng thời làm giảm accuracy trên ảnh test gốc.

6 Thảo luận

6.1 Đánh đổi độ chính xác, thời gian và khả năng giải thích

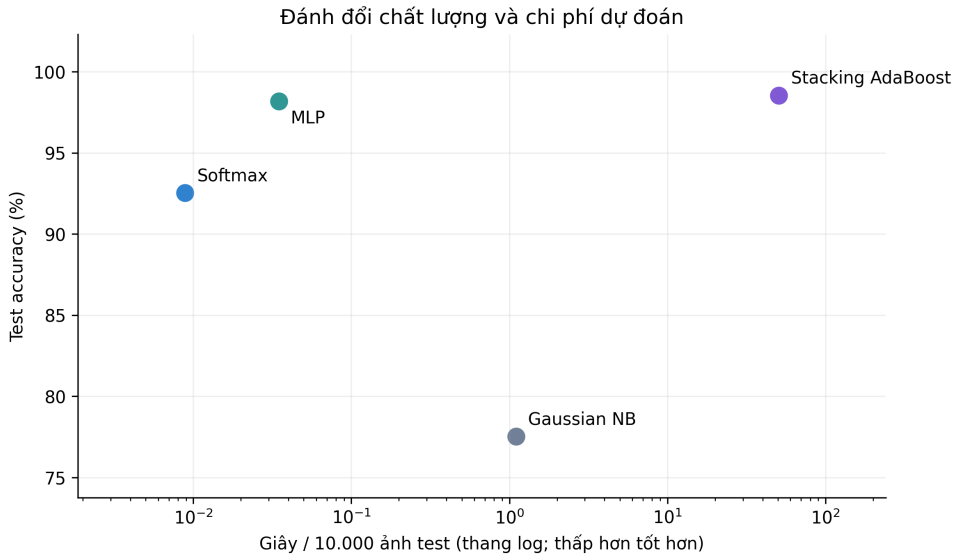
Bảng 9

Thời gian huấn luyện cấu hình cuối và dự đoán trên tập test (giây).

Phương pháp	Huấn luyện	Dự đoán 10.000 ảnh
Gaussian NB	0,410	1,103
Softmax	11,151	0,009
MLP	1980,533	0,035
Stacking AdaBoost	6513,241	50,715

Trong Bảng 9, Gaussian NB có thời gian huấn luyện ngắn nhất (0,410 giây), vì bước này chủ yếu tính các thống kê theo lớp. Softmax có thời gian dự đoán ngắn nhất (0,009 giây cho 10.000 ảnh). MLP tự viết đạt accuracy 98,19% với thời gian dự đoán 0,035 giây. So với MLP, Stacking AdaBoost tăng accuracy thêm 0,35 điểm phần trăm, lên 98,54%, và mất 50,715 giây để dự đoán cùng tập test. Khoảng thời gian này gồm bước tính xác suất từ ba mô hình cơ sở và bước dự đoán của tầng AdaBoost.

Gaussian NB ước lượng xác suất tiên nghiệm của mỗi lớp, đồng thời tính trung bình và phương sai của từng pixel trong lớp đó. Với Softmax, tích giữa giá trị pixel và trọng số tương ứng cho biết pixel đó đóng góp bao nhiêu vào logit của một lớp. Ở MLP, đầu vào đi qua nhiều lớp ẩn. Với stacking, dự đoán cuối dựa trên xác suất từ ba mô hình cơ sở. Vì vậy, để giải thích dự đoán của hai phương pháp này, cần xem xét cả các bước trung gian.



Hình 16

Accuracy và thời gian dự đoán toàn bộ 10.000 ảnh test. Trục thời gian dùng thang log.

7 Kết luận

Trong bốn phương pháp chính, MLP tự viết đạt accuracy 98,19% và F1 macro 0,9818 trên tập test. Stacking AdaBoost đạt accuracy cao nhất (98,54%). So với soft voting, stacking giảm ròng 2 lỗi trên 10.000 ảnh test và cần thêm bước tạo đặc trưng OOF và huấn luyện tầng meta.

Trong thí nghiệm mở rộng, CNN_FC128 đạt accuracy 98,70%, cao hơn MLP260 1,13 điểm phần trăm. Hai mô hình có số tham số gần nhau nhưng CNN cần nhiều thời gian huấn luyện hơn. Trong thí nghiệm 15 epoch, augmentation giúp cả hai mô hình đạt accuracy cao hơn ở các mức dịch và xoay lớn nhất đã thử, đồng thời làm giảm accuracy trên tập test gốc.

Phân công công việc

Bảng 10
Phân công công việc của các thành viên.

Thành viên	MSSV	Phần việc thực tế
Lê Mỹ Tài	24110235	EDA, cài đặt Softmax, stacking, CNN cơ bản, demo, slide và báo cáo
Trần Hùng Luyện	24110201	Cài đặt dữ liệu, EDA, Gaussian NB, MLP, slide và báo cáo

Khai báo hỗ trợ AI. Nhóm sử dụng ChatGPT để hỗ trợ tìm hiểu nguyên lý các thuật toán, đề xuất phương án thiết lập thực nghiệm và tổ chức lưu cấu hình, mô hình, kết quả để có thể tái sử dụng. ChatGPT cũng hỗ trợ xây dựng bố cục, biên tập nội dung và dàn trang báo cáo bằng LaTeX. Nhóm kiểm tra lại mã nguồn, công thức, kết quả thực nghiệm và nội dung trước khi nộp.

Tài liệu tham khảo

[1] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006. Bản PDF.

[2] S. Theodoridis và K. Koutroumbas. *Pattern Recognition*, ấn bản thứ hai. Academic Press, 2003. Trang nhà xuất bản.

[3] *ML-From-Scratch*. Mã nguồn tham khảo trong gói `ML-From-Scratch-master.zip`. Mã nguồn GitHub.

[4] *MLfromscratch*. Mã nguồn tham khảo trong gói `MLfromscratch-master.zip`. Mã nguồn GitHub.

[5] J. Zhu, H. Zou, S. Rosset và T. Hastie. *Multi-class AdaBoost*. Statistics and Its Interface, 2(3), 349–360, 2009. Bản PDF.